

# Gestion des feuilles de présence

Le but de ce projet est de réaliser une application de gestion des feuilles de présence.

L'application devra comporter une partie front en React et une partie back en Spring. Elle devra respecter les exigences énoncées ci-dessous.

Le projet est à réaliser en binôme ou par groupe de trois. Les binômes doivent réaliser le module de base et le module de gestion des droits d'accès. Les groupes de trois doivent réaliser le module de base, le module de gestion des droits d'accès et le module d'émargement.

Une présentation du projet d'une durée de 15 minutes suivies de 5 minutes de question est à réaliser lors de la dernière séance. Le code de l'application doit être déposé sur un dépôt git. L'intervenant doit être invité en tant que développeur sur ce dépôt.

## Module de base

Avec ce module, il s'agit de pouvoir effectuer une gestion de base des feuilles de présence d'une formation. Il faut pouvoir gérer des groupes, gérer des étudiants, gérer des agendas et gérer des feuilles de présence.

Un groupe est identifié par un nom. Il faut pouvoir afficher la liste des groupes, créer un groupe, supprimer un groupe.

Un étudiant est identifié par un numéro et possède un nom, un prénom et un email. Il faut pouvoir afficher la liste des étudiants, créer un étudiant, mettre à jour les données d'un étudiant, supprimer un étudiant. Il faut pouvoir inscrire un étudiant dans un groupe. Chaque étudiant est inscrit dans au plus un groupe.

Un agenda est identifié par un nom et est associé à un ensemble de séances. Il faut pouvoir associer un agenda à un groupe. Une séance possède une date, un horaire de début, un horaire de fin, le nom d'un intervenant et un intitulé. Il faut pouvoir importer un ensemble de séances dans un agenda à partir d'un fichier au format ICS. Il faut pouvoir afficher la liste des séances d'un agenda.

Une feuille de présence correspond à l'ensemble des séances pour un étudiant et un mois. Il faut pouvoir afficher une feuille de présence en vue de l'imprimer.

## Module de gestion des droits d'accès

Avec ce module, il s'agit de pouvoir gérer des droits d'accès différenciés à l'application en fonction du rôle de l'utilisateur. Il faut pouvoir gérer des rôles et des utilisateurs.

Un rôle est identifié par un nom. Il faut pouvoir afficher la liste des rôles, créer un rôle, supprimer un rôle.

Un utilisateur est identifié par un email et possède un nom et un prénom. Il faut pouvoir afficher la liste des utilisateurs, créer un utilisateur, modifier un utilisateur, supprimer un utilisateur. Il faut pouvoir associer un ou plusieurs rôles à un utilisateur.

En première approche, on considère que l'application contient deux rôles : gestionnaire et étudiant. Le rôle étudiant permet uniquement d'afficher une feuille de présence en vue de l'imprimer. Le rôle gestionnaire permet d'accéder à l'ensemble des fonctionnalités du module de base.

## Module d'émargement

Avec ce module, vous proposerez une solution pour permettre aux étudiants et aux intervenants d'émarguer de façon numérique à chaque séance.

## Exigences à respecter

### Front

La partie front du projet doit :

- Être développée avec React
- Proposer un look et une ergonomie agréables
- Gérer les erreurs potentielles
  - Validations avancées des saisies
    - Yup ou équivalent
  - Erreurs réseau
  - Latence des requêtes (afficher un message 'loading')
  - Erreurs venant du serveur
  - etc.
- Être développée comme une single page application en utilisant un routeur
- Utiliser des formulaires avancés
  - Formik ou équivalent
- Mettre en œuvre une séparation des préoccupations
  - Services

- Composants ne faisant qu'une préoccupation
    - Soit affichage
    - Soit collecte / envoi des données
- Mettre en œuvre des Hooks custom
- Mettre en œuvre des requêtes avec Fetch
  - Dans un service
  - Avec des hooks custom si nécessaire
  - Gérant la latence, les erreurs, etc. (retourner des booléens indiquant l'état de la requête, voir cours ARI1)
  - Axios ou autre librairies non autorisées
- [Optionnel] Gérer un cache de requêtes avec @tanstack/react-query
- Le projet doit démarrer avec la commande 'npm run dev'

## Back

Le partie back du projet doit :

- Pouvoir être lancé avec uniquement la commande Gradle ou Maven
- Pouvoir être exécuté **sans** utiliser un container comme docker ou similaire

## Organisation du projet

- Utilisation de gitlab-etu.fil.univ-lille.fr ou gitlab.univ-lille.fr uniquement
- Nom du dépôt :
  - MIAGE\_M1\_2026\_nom1\_prenom1\_nom2\_prenom\_2
- Ajouter votre intervenant en tant que 'developper'
- Un seul dépôt contenant les deux projets
  - Front : 'front'
  - Back : 'back'
- A propos des commits dans le projet :
  - Il doit y avoir des commits de la part de chaque membre de l'équipe
  - Il doit y avoir plusieurs commits à chaque séance
  - Il faut utiliser le mécanisme des branches :
    - Lors de l'ajout d'une fonctionnalité :
      - Création d'une nouvelle branche
      - Développement de la fonction dans cette branche
      - Quand la fonction est ok, merge de la branche dans la branche principale
-